



CS4001NI Programming

30% Individual Coursework

2024 Spring

Student Name: Abinash kumar sah

London Met ID: 23056140

College ID: NP01NT4S240058

Group: N11

Assignment Due Date: Friday, August 23, 2024

Assignment Submission Date: Friday, August 23, 2024

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Contents

1. Introduction	5
1.2 Introducing Project Content	5
1.3 Tools used	5
1.3.1 Blue j	5
1.3.2 Ms word	6
1.3.3 Draw.io	6
2. Class Diagram	7
2.1 Store GUI	7
2.2 Store Class	8
2.3 Department Class	9
2.4 Retailer class	10
3. Pseudocode	11
4. Method Description	15
4.1 Button Methods	15
4.2 Add Retailer Button	15
4.3 Calculate Discount Price Button	16
4.4 Set Loyalty Point Button	16
4.5 Remove Product Button	16
4.6 Display Button	16
4.7 Clear Button	17
5. Testing	17
5.1 Test 1: (Computing&Running using command prompt)	17
5.2 Test 2: Evidence for Functionality	18
5.2.1 Add Department	18
5.2.2 Add Retailer	18
5.2.3 Calculate Discount Price	19
5.2.4 Set loyalty point	19
6. Error Detection and Correction	20
6.1 syntax Error	20
6.2 Semantic Error	21
6.3 Logical error	22
7. Appendix	22

8. Conclusion	32
9. Bibliography	32

Table of figure

Figure 1: Ms word	6
Figure 2:Draw.io.....	6
Figure 3:Store GUI.....	7
Figure 4:Store class	8
Figure 5:Department class	9
Figure 6:Retailer class	10
Figure 7:figure of test 1	17
Figure 8:figure of add department.....	18
Figure 9:figure of add retailer.....	18
Figure 10:figure of calculate discount price.....	19
Figure 11:figure of set loyalty point	19

1. Introduction

This is CourseWork2 for Java programming language in which we are instructed to simulate IRL (In Real Life) scenario of Store GUI process by using OOP (object oriented programming) through BlueJ. This will help to check the Store's detail with the implementation of GUI.

1.2 Introducing Project Content

This project revolves around creating a Graphical User Interface (GUI), which is a way to visually represent an object's structure. The object in question has been passed down through the classes of store GUI building upon previous coursework. The entire project was executed in the BlueJ development environment. I had the assistance of Microsoft Word to write the report for this project, and I have included details about the tools used in the process.

1.3 Tools used

1.3.1 Blue j



Michael Kölling introduced the instructional language and environment called Blue during his doctoral research in the late 1990s. The Java adaptation of Blue, known as BlueJ, was initially released in 1999. Subsequently, the ongoing development and maintenance of BlueJ have been possible through the generous support of Sun Microsystems and, more recently, Oracle. Initially based in Australia, the team expanded to the UK and later to Denmark after Michael's relocation. Currently, all team members operate from the Computing

Education Research Group at the University of Kent, having initially moved to the UK as the Danish team and later as the Australian team did (BlueJ, n.d.). BlueJ serves as the Integrated Development

Environment (IDE) for the Java programming language. It enhances the user interface for Java project creation and coding, contributing to a more precise development experience (Techopedia, 2020)

1.3.2 Ms word



Figure 1: Ms word

In 1983, Microsoft introduced Microsoft Word, a word processor software that has since become the most widely utilized in its category. This software serves the purpose of crafting high-quality documents, including but not limited to letters, reports, resumes, and more. Microsoft Word not only facilitates the creation of new documents but also provides tools for the editing and modification of existing ones. Documents created with Microsoft Word are typically saved with a doc extension. While Microsoft Word is a component of the Microsoft Office suite, it is also available for individual purchase and is compatible with both Windows and macOS operating systems. As of the latest information available, the 2019 version is the most recent iteration of Microsoft Word (geeksforgets, 2022)

1.3.3 Draw.io



Figure 2:Draw.io

Using the freely accessible diagramming tool Draw.io, individuals can create and share diagrams directly through their web browsers. The online application is designed to facilitate audits and seamlessly integrate with Atlassian's Jira and Confluence. Additionally, it is compatible with file storage services like Dropbox, as well as productivity suites such as G Suite/Google Drive (Source: (getapp.n.d, n.d.))

2. Class Diagram

A class diagram serves as a static blueprint in object-oriented programming, illustrating relationships between classes. It also effectively showcases the organizational structure of a system. In the context of business applications, these diagrams elucidate class associations, providing a visual representation of the system's architecture.

2.1 Store GUI

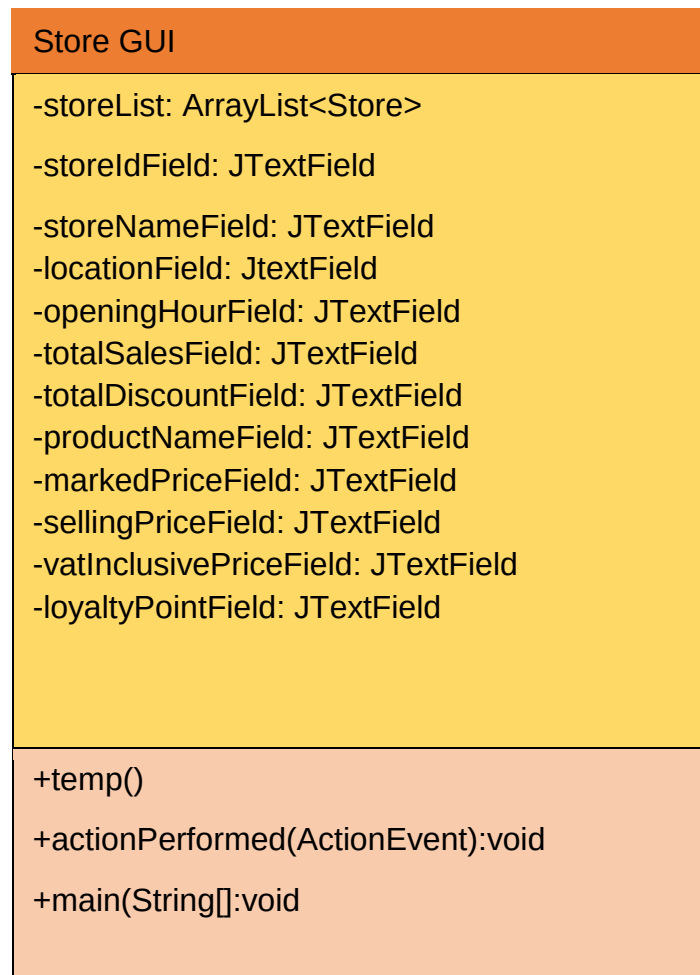


Figure 3:Store GUI

2.2 Store Class

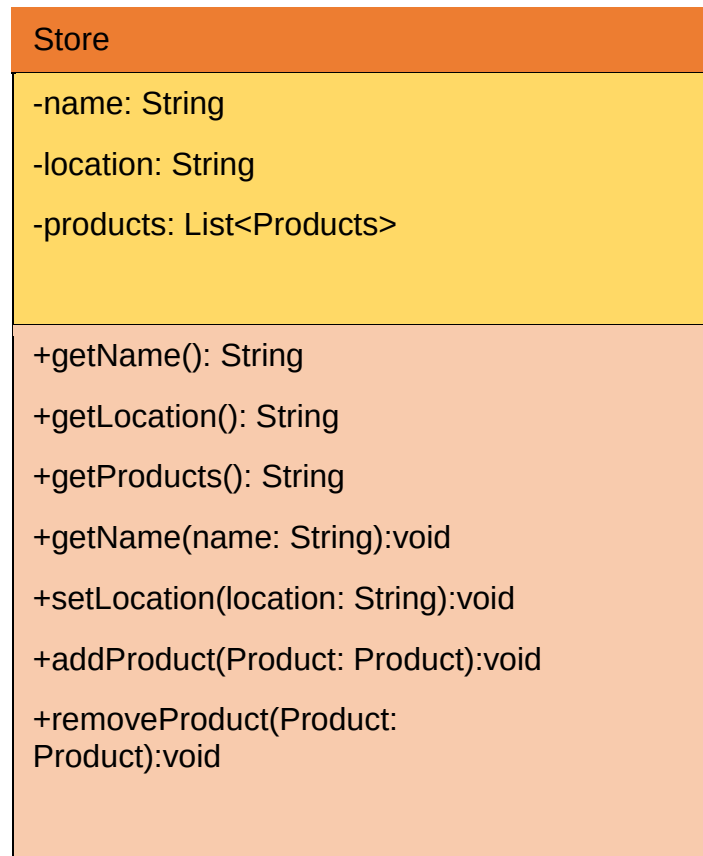


Figure 4:Store class

2.3 Department Class

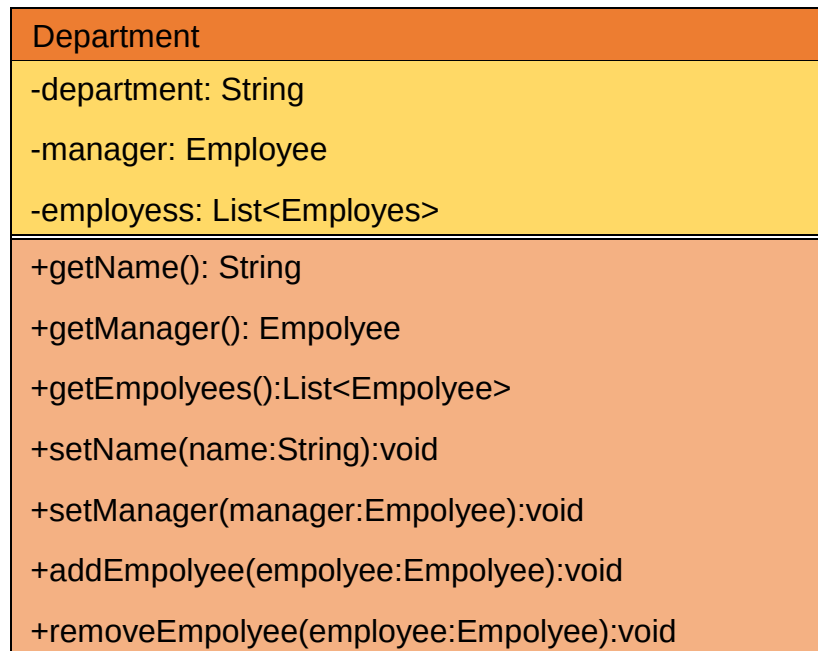


Figure 5:Department class

2.4 Retailer class

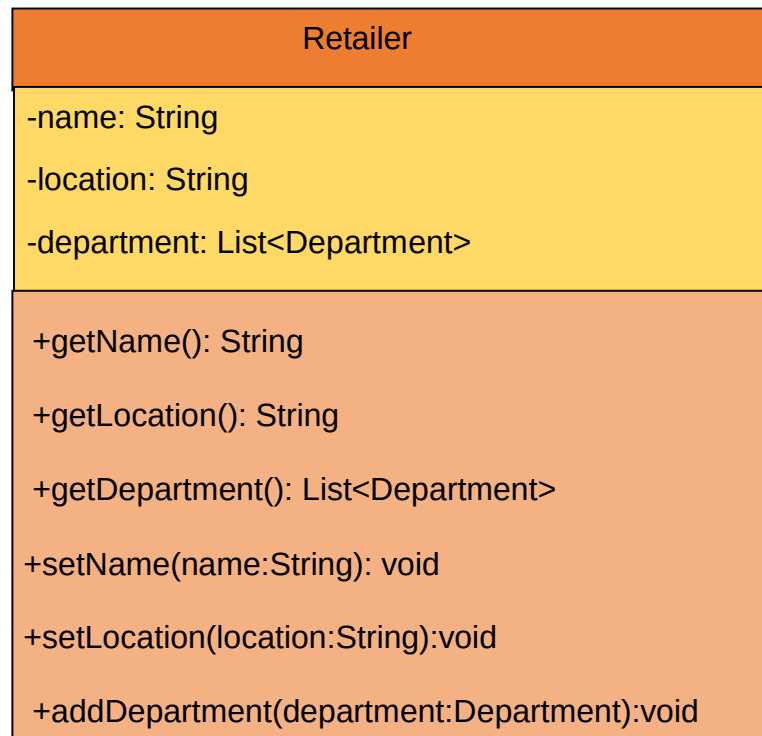


Figure 6:Retailer class

3. Pseudocode

Pseudo-code serves as an informal method for illustrating programming logic without the constraints of specific syntax or technological details. It functions as a preliminary sketch or blueprint for a program, capturing its essential structure. Although pseudocode streamlines the program's logic, it excludes supplementary details. System designers employ pseudocode to ensure that programmers understand the software project's specifications and can align their code accordingly (economytimes, n.d.).

Store GUI

Class StoreGUI

Initialize Frame

Create JFrame titled "Store Management"

Set default close operation to EXIT_ON_CLOSE

Set size to 800x600

Set layout to BorderLayout

Initialize Components

Create JTextField for store ID, store name, location, opening hour, total sales, total discount, product name, marked price, selling price, VAT inclusive price, loyalty point

Create JCheckBox for payment online

Create JComboBox for purchased year with options "2020", "2021", "2022", "2023"

Create JButton for adding department, adding retailer, calculating discount, setting loyalty points, removing product, displaying store list, and clearing fields

Initialize ArrayList to store Store objects

Create and Configure Main Panel

Create JPanel with GridBagLayout

Set border with empty padding

Define GridBagConstraints for layout

Add text fields and labels to main panel

Add "Store ID", "Store Name", "Location", etc., with corresponding text fields

Create Payment Options Panel

Create JPanel with GridBagLayout

Set border with titled border "Payment Options"

Add JCheckBox and JComboBox for payment options

Add Payment Options Panel to Main Panel

Create Button Panel

Create JPanel with GridLayout for buttons

Add all buttons to the panel

Add Main Panel and Button Panel to Frame

Define Action Listeners

Add Department Button Listener

Try

Parse input from text fields

Create Department object

Add to store list

Show success message

Catch NumberFormatException

Show error message for invalid numbers

Add Retailer Button Listener

Try

Parse input from text fields

Create Retailer object

Add to store list

Show success message

Catch NumberFormatException

Show error message for invalid numbers

Calculate Discount Button Listener

Try

Parse store ID

Search store list for matching Department

Calculate discount price

Update selling price field

Show success message

Catch NumberFormatException

Show error message for invalid Store ID

Set Loyalty Point Button Listener

Try

Parse store ID

Search store list for matching Retailer

Calculate and set loyalty points

Update loyalty point field

Show success message

Catch NumberFormatException

Show error message for invalid Store ID

Remove Product Button Listener

Try

Parse store ID

Search store list for matching Retailer

Remove product

Show success message

Catch NumberFormatException

Show error message for invalid Store ID

Display Button Listener

Build a string with all store information from store list

Show the list in a dialog

Clear Button Listener

Clear all text fields and reset options

Main Method

Invoke SwingUtilities to create and show GUI on Event Dispatch Thread

End Class

4. Method Description

4.1 Button Methods

1. Add Department Button

- Method Name:
AddDepartmentButtonListener.actionPerformed(ActionEvent e)
- Description:
 - Retrieves values from input fields related to a department.
 - Creates a new Department object using the retrieved values.
 - Adds the newly created department to the stores list.
 - Displays a success message if the department is added successfully.
 - Handles invalid input by showing an error message.

4.2 Add Retailer Button

- Method Name: AddRetailerButtonListener.actionPerformed(ActionEvent e)
- Description:
 - Retrieves values from input fields related to a retailer.
 - Creates a new Retailer object using the retrieved values.
 - Adds the newly created retailer to the stores list.
 - Displays a success message if the retailer is added successfully.
 - Handles invalid input by showing an error message.

4.3 Calculate Discount Price Button

- Method Name: CalculateDiscountPriceButtonListener.actionPerformed(ActionEvent e)
- Description:
 - Retrieves the store ID from the input fields.
 - Searches for a Department in the stores list with the matching store ID.
 - If found, calculates the discount price based on the marked price entered.
 - Displays the calculated selling price in the respective field.
 - Shows a message with the discount price.
 - Displays an error message if the store ID is not found or is not a Department.
 - Handles invalid input by showing an error message.

4.4 Set Loyalty Point Button

- Method Name: SetLoyaltyPointButtonListener.actionPerformed(ActionEvent e)
- Description:
 - Retrieves the store ID and other relevant data from the input fields.
 - Searches for a Retailer in the stores list with the matching store ID.
 - If found, calculates loyalty points if payment is online.
 - Displays a message indicating that loyalty points were calculated or an error message if payment is not online.
 - Displays an error message if the store ID is not found or is not a Retailer.
 - Handles invalid input by showing an error message.

4.5 Remove Product Button

- Method Name: RemoveProductButtonListener.actionPerformed(ActionEvent e)
- Description:
 - Retrieves the store ID from the input fields.
 - Searches for a Department in the stores list with the matching store ID.
 - If found, removes a product from the department.
 - Displays a success message if the product is removed.
 - Shows an error message if the store ID is not found or is not a Department.
 - Handles invalid input by showing an error message.

4.6 Display Button

- Method Name: DisplayButtonListener.actionPerformed(ActionEvent e)
- Description:
 - Iterates through the stores list and collects information for each Department and Retailer.
 - Constructs a string with the collected information.
 - Displays the compiled store information in a message dialog.
 - Shows a message indicating that no store information is available if the list is empty.

4.7 Clear Button

- Method Name: ClearButtonListener.actionPerformed(ActionEvent e)
- Description:
 - Clears all input fields and resets the state of the checkboxes and combo boxes to their default values.
 - Ensures that all input fields are empty and all selections are reset for a fresh start.

5. Testing

5.1 Test 1: (Computing&Running using command prompt)

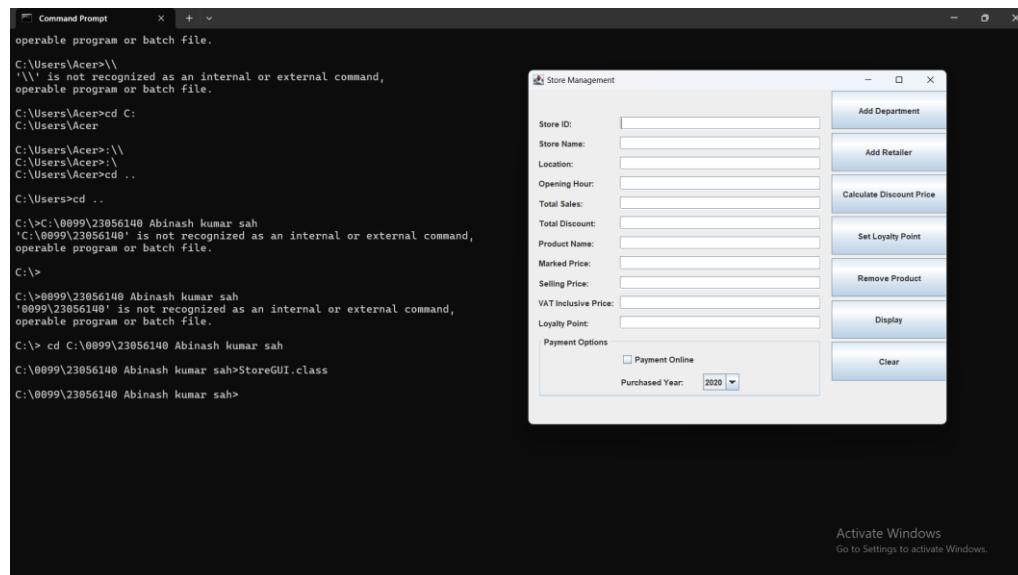


Figure 7:figure of test 1

5.2 Test 2: Evidence for Functionality

5.2.1 Add Department

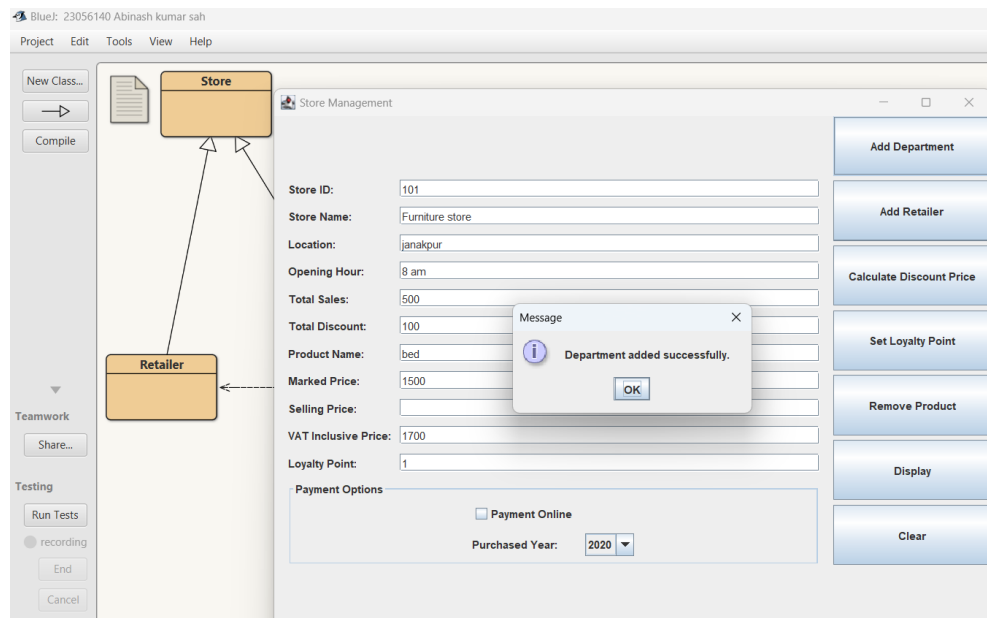


Figure 8:figure of add department

5.2.2 Add Retailer

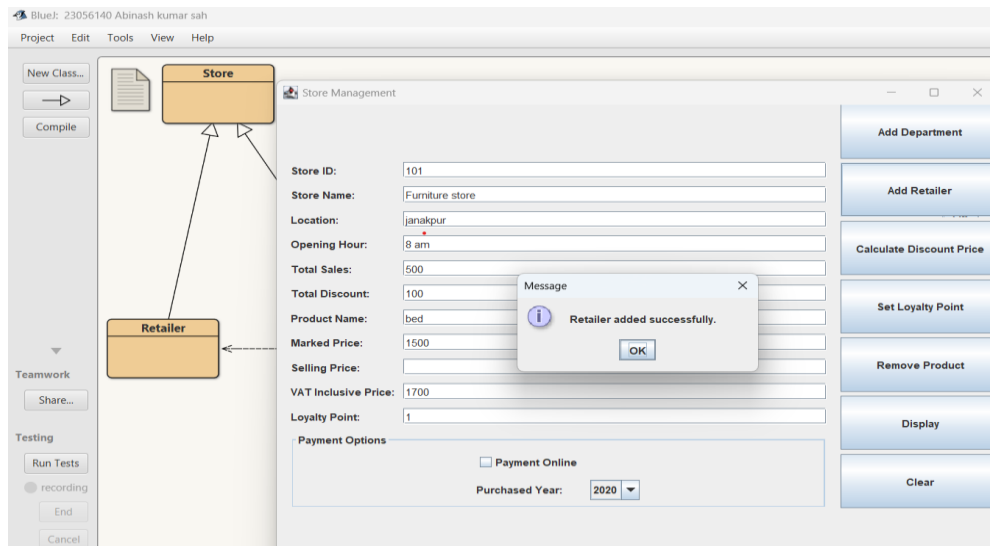


Figure 9:figure of add retailer

5.2.3 Calculate Discount Price

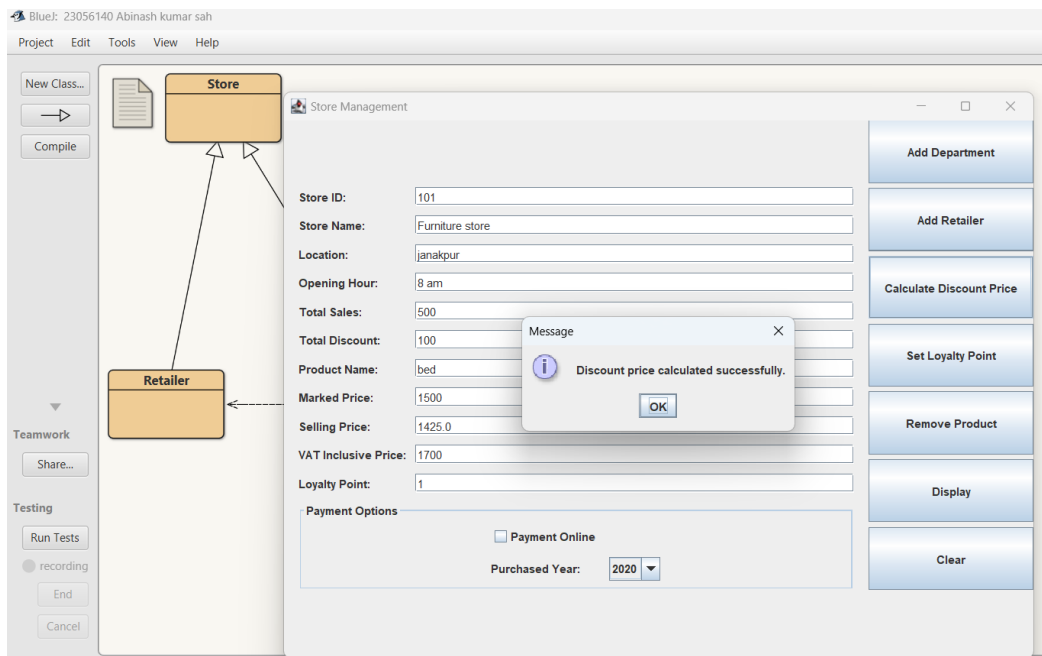


Figure 10:figure of calculate discount price

5.2.4 Set loyalty point

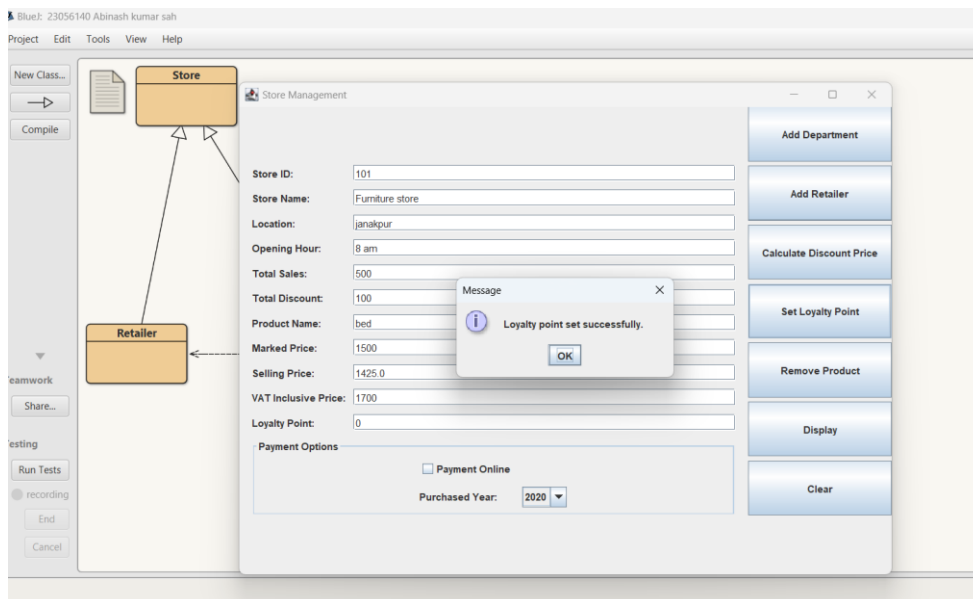


Figure 11:figure of set loyalty point

5.3 Test 3: Handling invalid Store ID

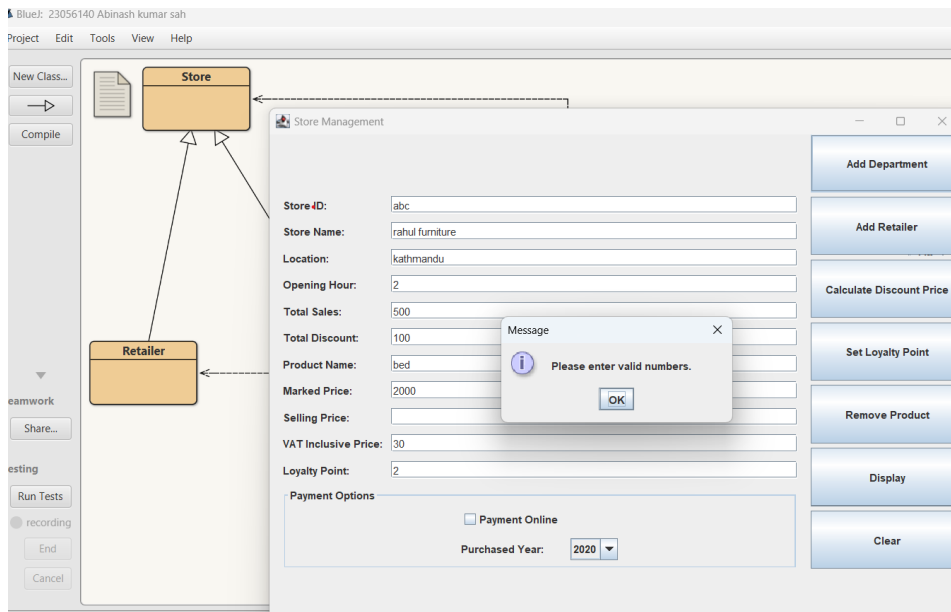


Figure 12: Handling invalid Store ID

6. Error Detection and Correction

6.1 syntax Error

A syntax error occurs when the code in a program doesn't adhere to the specific rules of the programming language. Because computer programs require precise syntax for proper compilation, any code elements that deviate from these rules result in a syntax error. (Rouse, Techopedia 2017) Error Detection: Missing semi colon (;)

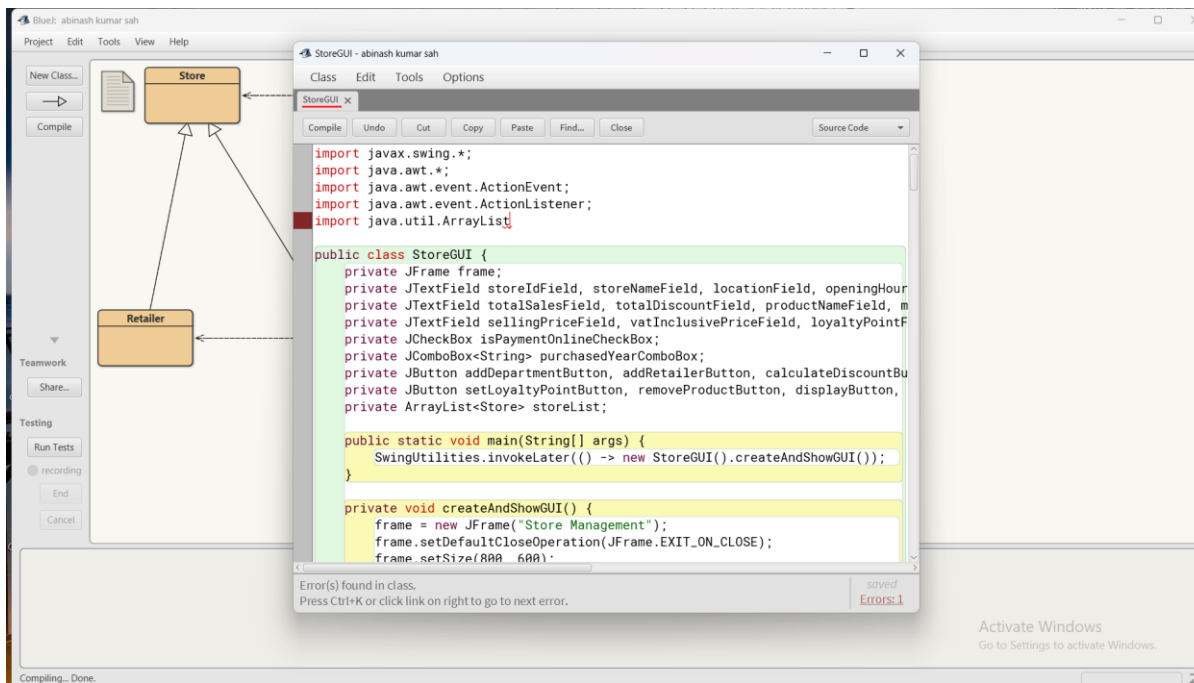
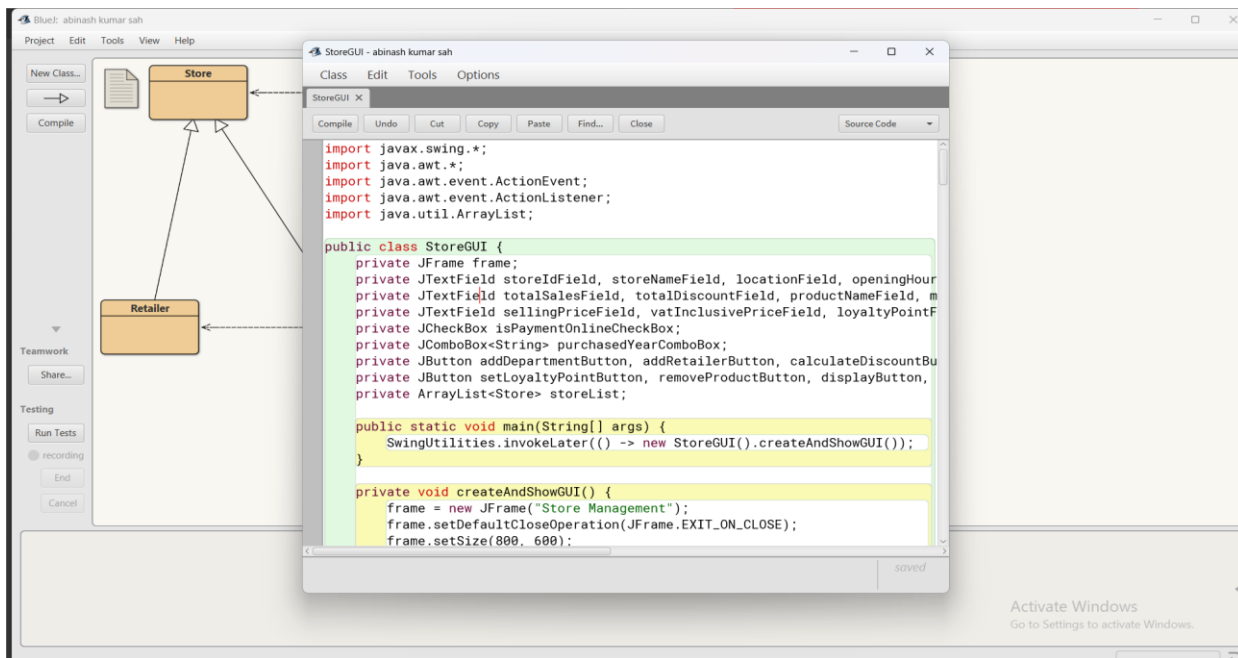


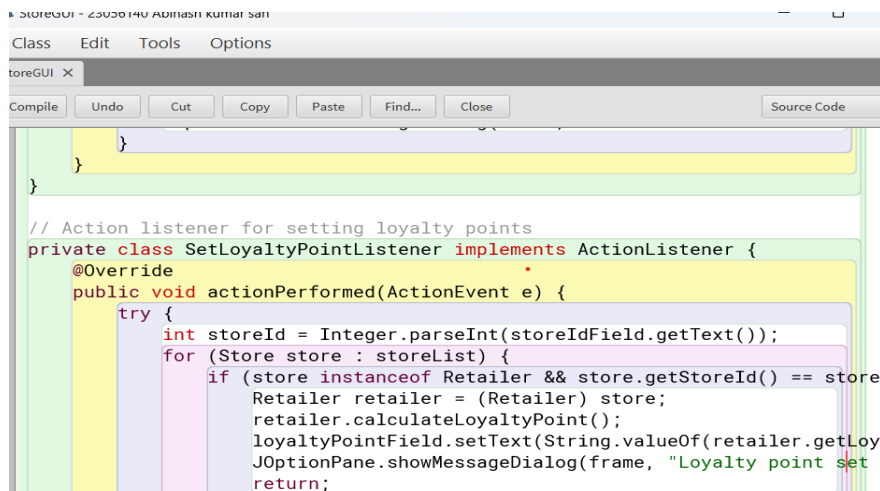
Figure 13: Screenshot of syntax error

6.2 Semantic Error

A semantic error in programming occurs when there is a misconception about the meaning or impact of a certain element in the code of a programming language. (encyclopedia, n.d.) In Java programming, a semantic error occurs when the data type of a variable is mistakenly changed. For instance, designating the type as 'int' was an error; it should have been 'String.' Despite this mistake, the program ran smoothly after correcting the data type from 'int' to 'String.'



6.3 Logical error



7.Appendix

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
```

```
import java.util.ArrayList;

public class StoreGUI {
    private JFrame frame;
    private JTextField storeIdField, storeNameField, locationField, openingHourField;
    private JTextField totalSalesField, totalDiscountField, productNameField,
markedPriceField;
    private JTextField sellingPriceField, vatInclusivePriceField, loyaltyPointField;
    private JCheckBox isPaymentOnlineCheckBox;
    private JComboBox<String> purchasedYearComboBox;
    private JButton addDepartmentButton, addRetailerButton, calculateDiscountButton;
    private JButton setLoyaltyPointButton, removeProductButton, displayButton, clearButton;
    private ArrayList<Store> storeList;

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new StoreGUI().createAndShowGUI());
    }

    private void createAndShowGUI() {
        frame = new JFrame("Store Management");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(800, 600);

        frame.setLayout(new BorderLayout());

        // Create and add the main panel with GridBagLayout
        JPanel mainPanel = new JPanel();
        mainPanel.setLayout(new GridBagLayout());
        mainPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
        GridBagConstraints gbc = new GridBagConstraints();
```

```
gbc.insets = new Insets(5, 5, 5, 5);
gbc.fill = GridBagConstraints.HORIZONTAL;

// Initialize components
storeIdField = new JTextField(20);
storeNameField = new JTextField(20);
locationField = new JTextField(20);
openingHourField = new JTextField(20);
totalSalesField = new JTextField(20);
totalDiscountField = new JTextField(20);
productNameField = new JTextField(20);
markedPriceField = new JTextField(20);
sellingPriceField = new JTextField(20);
vatInclusivePriceField = new JTextField(20);
loyaltyPointField = new JTextField(20);

isPaymentOnlineCheckBox = new JCheckBox("Payment Online");
purchasedYearComboBox = new JComboBox<>(new String[]{"2020", "2021", "2022",
"2023"});

addDepartmentButton = new JButton("Add Department");
addRetailerButton = new JButton("Add Retailer");
calculateDiscountButton = new JButton("Calculate Discount Price");
setLoyaltyPointButton = new JButton("Set Loyalty Point");
removeProductButton = new JButton("Remove Product");
displayButton = new JButton("Display");
clearButton = new JButton("Clear");

storeList = new ArrayList<>();
```



```

// Add components to the main panel with GridBagLayout
addComponent(mainPanel, new JLabel("Store ID:"), storeIdField, 0, 0, 1);
addComponent(mainPanel, new JLabel("Store Name:"), storeNameField, 0, 1, 1);
addComponent(mainPanel, new JLabel("Location:"), locationField, 0, 2, 1);
addComponent(mainPanel, new JLabel("Opening Hour:"), openingHourField, 0, 3, 1);
addComponent(mainPanel, new JLabel("Total Sales:"), totalSalesField, 0, 4, 1);
addComponent(mainPanel, new JLabel("Total Discount:"), totalDiscountField, 0, 5, 1);
addComponent(mainPanel, new JLabel("Product Name:"), productNameField, 0, 6, 1);
addComponent(mainPanel, new JLabel("Marked Price:"), markedPriceField, 0, 7, 1);
addComponent(mainPanel, new JLabel("Selling Price:"), sellingPriceField, 0, 8, 1);
addComponent(mainPanel, new JLabel("VAT Inclusive Price:"), vatInclusivePriceField,
0, 9, 1);
addComponent(mainPanel, new JLabel("Loyalty Point:"), loyaltyPointField, 0, 10, 1);

gbc.gridx = 1;
gbc.gridy = 6;
mainPanel.add(isPaymentOnlineCheckBox, gbc);
gbc.gridy = 9;
mainPanel.add(purchasedYearComboBox, gbc);

// Button panel with GridLayout
JPanel buttonPanel = new JPanel();
buttonPanel.setLayout(new GridLayout(8, 1, 5, 5));
buttonPanel.add(addDepartmentButton);
buttonPanel.add(addRetailerButton);
buttonPanel.add(calculateDiscountButton);
buttonPanel.add(setLoyaltyPointButton);
buttonPanel.add(removeProductButton);
buttonPanel.add(displayButton);
buttonPanel.add(clearButton);

```

```
frame.add(mainPanel, BorderLayout.CENTER);
frame.add(buttonPanel, BorderLayout.EAST);

// Add action listeners
addDepartmentButton.addActionListener(new AddDepartmentListener());
addRetailerButton.addActionListener(new AddRetailerListener());
calculateDiscountButton.addActionListener(new CalculateDiscountListener());
setLoyaltyPointButton.addActionListener(new SetLoyaltyPointListener());
removeProductButton.addActionListener(new RemoveProductListener());
displayButton.addActionListener(new DisplayListener());
clearButton.addActionListener(new ClearListener());

frame.setVisible(true);
}
```

```
private void addComponent(JPanel panel, Component label, Component field, int gridx,
int gridy, int gridwidth) {
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.gridx = gridx;
    gbc.gridy = gridy;
    gbc.insets = new Insets(5, 5, 5, 5);
    gbc.anchor = GridBagConstraints.WEST;
    panel.add(label, gbc);

    gbc.gridx = gridx + 1;
    gbc.fill = GridBagConstraints.HORIZONTAL;
    gbc.weightx = 1.0;
    panel.add(field, gbc);
}
```

```

private class AddDepartmentListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            int storeId = Integer.parseInt(storeIdField.getText());
            String storeName = storeNameField.getText();
            String location = locationField.getText();
            String openingHour = openingHourField.getText();
            double totalSales = Double.parseDouble(totalSalesField.getText());
            double totalDiscount = Double.parseDouble(totalDiscountField.getText());
            String productName = productNameField.getText();
            double markedPrice = Double.parseDouble(markedPriceField.getText());

            Store store = new Department(storeId, storeName, location, openingHour,
totalSales, totalDiscount, productName, markedPrice);
            storeList.add(store);
            JOptionPane.showMessageDialog(frame, "Department added successfully.");
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(frame, "Please enter valid numbers.");
        }
    }
}

```

```

private class AddRetailerListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            int storeId = Integer.parseInt(storeIdField.getText());
            String storeName = storeNameField.getText();

```

```

String location = locationField.getText();
String openingHour = openingHourField.getText();
double totalSales = Double.parseDouble(totalSalesField.getText());
double totalDiscount = Double.parseDouble(totalDiscountField.getText());
int vatInclusivePrice = Integer.parseInt(vatInclusivePriceField.getText());
boolean isPaymentOnline = isPaymentOnlineCheckBox.isSelected();
String purchasedYear = (String) purchasedYearComboBox.getSelectedItem();

Store store = new Retailer(storeId, storeName, location, openingHour, totalSales,
totalDiscount, vatInclusivePrice, isPaymentOnline, purchasedYear);
storeList.add(store);
OptionPane.showMessageDialog(frame, "Retailer added successfully.");
} catch (NumberFormatException ex) {
OptionPane.showMessageDialog(frame, "Please enter valid numbers.");
}
}
}

private class CalculateDiscountListener implements ActionListener {
@Override
public void actionPerformed(ActionEvent e) {
try {
int storeId = Integer.parseInt(storeIdField.getText());
for (Store store : storeList) {
if (store instanceof Department && store.getStoreId() == storeId) {
Department department = (Department) store;
department.calculateDiscountPrice(true);
sellingPriceField.setText(String.valueOf(department.getSellingPrice()));
OptionPane.showMessageDialog(frame, "Discount price calculated
successfully.");
}
}
}
}
}

```

```

        return;
    }
}
JOptionPane.showMessageDialog(frame, "Department with this Store ID not
found.");
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(frame, "Please enter a valid Store ID.");
}
}
}
}

```

```

private class SetLoyaltyPointListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            int storeId = Integer.parseInt(storeIdField.getText());
            for (Store store : storeList) {
                if (store instanceof Retailer && store.getStoreId() == storeId) {
                    Retailer retailer = (Retailer) store;
                    retailer.calculateLoyaltyPoint();
                    loyaltyPointField.setText(String.valueOf(retailer.getLoyaltyPoint()));
                    JOptionPane.showMessageDialog(frame, "Loyalty point set successfully.");
                    return;
                }
            }
            JOptionPane.showMessageDialog(frame, "Retailer with this Store ID not found.");
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(frame, "Please enter a valid Store ID.");
        }
    }
}

```

```
}
```

```
private class RemoveProductListener implements ActionListener {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        try {  
            int storeId = Integer.parseInt(storeIdField.getText());  
            for (Store store : storeList) {  
                if (store instanceof Retailer && store.getStoreId() == storeId) {  
                    Retailer retailer = (Retailer) store;  
                    retailer.removeProduct();  
                    JOptionPane.showMessageDialog(frame, "Product removed successfully.");  
                    return;  
                }  
            }  
            JOptionPane.showMessageDialog(frame, "Retailer with this Store ID not found.");  
        } catch (NumberFormatException ex) {  
            JOptionPane.showMessageDialog(frame, "Please enter a valid Store ID.");  
        }  
    }  
}
```

```
private class DisplayListener implements ActionListener {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        StringBuilder sb = new StringBuilder();  
        for (Store store : storeList) {  
            sb.append(store.toString()).append("\n\n");  
        }  
    }  
}
```

```
        JOptionPane.showMessageDialog(frame, sb.toString(), "Store List",
JOptionPane.INFORMATION_MESSAGE);
    }
}
```

```
private class ClearListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        storeIdField.setText("");
        storeNameField.setText("");
        locationField.setText("");
        openingHourField.setText("");
        totalSalesField.setText("");
        totalDiscountField.setText("");
        productNameField.setText("");
        markedPriceField.setText("");
        sellingPriceField.setText("");
        vatInclusivePriceField.setText("");
        loyaltyPointField.setText("");
        isPaymentOnlineCheckBox.setSelected(false);
        purchasedYearComboBox.setSelectedIndex(0);
    }
}
}
```

8. Conclusion

In conclusion, the provided code is a Java Swing application for managing a list of teachers, specifically lecturers and tutors. It utilizes a graphical user interface (GUI) to input teacher information such as ID, name, address, working type, employment status, working hours, department (for lecturers), years of experience, graded score (for lecturers), salary (for tutors), specialization, academic qualifications, and performance index.

The GUI consists of text fields for input, labels for field descriptions, and buttons for various actions like adding lecturers or tutors, grading assignments, setting salary, removing tutors, displaying teacher information, and clearing input fields.

The code defines classes for `StoreGUI`, `Retailer`, and `Department`, where `StoreGUI` extends `JFrame` and serves as the main GUI class. It implements action listeners for different buttons to handle user interactions and perform corresponding actions such as adding new store, grading assignments, setting salaries, and displaying teacher information. Overall, this code provides a basic framework for managing teacher information through a user-friendly interface, allowing users to perform various tasks related to teachers efficiently.

9. Bibliography

- geeksforgets. (2022). *geeksforgets*. Retrieved from <https://www.geeksforgeeks.org/python-programming-language-tutorial/>
- getapp.n.d. (n.d.). *getapp.n.d.* Retrieved from getapp: <https://www.getapp.com/>
- Techopedia. (2020). *www.techopedia.com*. Retrieved from www.techopedia.com: <https://www.techopedia.com/>